

Mock 2 for Milestone -2 Assessment

#Details

Q1: React Basics – ProductCard

Details

- Functional component that displays a product.
- Receives **props**: title, price, discount.
- Calculates **final price = price – discount**.

Features

- Reusable product card.
- Shows price breakdown.
- Dynamic props passed from parent.

Q2: Controlled + Uncontrolled – LoginForm

Details

- Login form capturing **username (controlled)** and **password (uncontrolled)**.
- On submit, logs values to console.

Features

- Controlled input using useState.
- Uncontrolled input using useRef.
- Form submission logging.

Q3: Class Component + Lifecycle – UserStatus

Details

- Class component showing Fetching user status....
- Updates to Active User after 2 seconds.
- Accepts userId as **PropTypes**.

Features

- Class component lifecycle (componentDidMount).
- Prop validation with PropTypes.
- Basic styling.

Q4: React Router + API Integration – UserDetails

Details

- Fetches user data from backend by id route parameter.
- Route: /users/:id.
- Example JSON server endpoint: http://localhost:4000/users/:id.

Features

- React Router route parameter.
- Fetch user data using fetch.
- Displays name and email.

Q5: Reusability Using HOC – ResponsiveDisplay

Details

- Displays **current window width**.
- Updates dynamically on resize.
- Uses **Higher-Order Component (HOC)**.

Features

- Reusable window width logic.
- Wraps any component.

Q6: Formik + Yup Validation – LoginFormik

Details

- Login form with **Formik** and **Yup validation**.
- Email format validated.
- Password minimum 6 characters.

Features

- Clean form UI.
- Validation messages.
- Logs values on submit.

File Structure

/milestone2

/node_modules

/public

index.html

favicon.ico

/src

/components

ProductCard.js # Q1

LoginForm.js # Q2

UserStatus.js # Q3

UserDetails.js # Q4

DisplayWidth.js # Q5 (HOC child)

withWindowWidth.js # Q5 (HOC wrapper)

ResponsiveDisplay.js # Q5

LoginFormik.js # Q6

App.js

index.jsr

db.json

package.json

package-lock.json

Execution

In terminal 1

>npm install

npm install react-router-dom formik yup prop-types

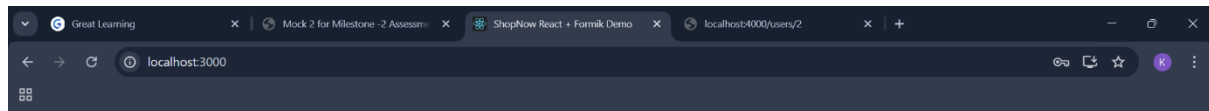
>npm start

In terminal 2

> npm install -g json-server

```
> json-server --watch db.json --port 4000
```

Output



ShopNow React + Formik Demo

Laptop
Price: Rs1000
Discount: Rs200
Final Price: Rs800

User 101: Active User

Window width: 1360px



Output for Question 1,2,3,5

ShopNow React + Formik Demo
Laptop
Price: Rs1000
Discount: Rs200
Final Price: Rs800

User 101: Active User

Window width: 643px

Elements

Styles

Console

Download the React DevTools for a better development experience:
<https://react.dev/link/react-devtools>

[DOM] Input elements should have autocomplete attributes (suggested: "current-password"): (index):1
(More info: <https://goo.gl/9p2vKq>) <input placeholder="Password" type="password">

[DOM] Input elements should have autocomplete attributes (suggested: "current-password"): (index):1
(More info: <https://goo.gl/9p2vKq>) <input placeholder="Password" type="password" value="123456" name="password">

WebSocket connection to 'ws://localhost:3000/ws' failed: WebSocketClient.js:13

Username: Kiranpp Loginform.js:9

Password: Kiran123 Loginform.js:10

Object Loginformik.js:11

Output for Question 2,6 look at console

A screenshot of a web browser window. The address bar shows 'localhost:4000/users/'. The page content displays a JSON array of three user objects, formatted by a 'Pretty-print' tool. The users are: 1. id: '1', name: 'kiran', email: 'kiran@example.com'; 2. id: '2', name: 'pavan', email: 'pavan@example.com'; 3. id: '3', name: 'pamidi', email: 'pamidi@example.com'. The browser's taskbar at the bottom shows various application icons and the system clock indicating 16:39 on 21-11-2025.

```
[
  {
    "id": "1",
    "name": "kiran",
    "email": "kiran@example.com"
  },
  {
    "id": "2",
    "name": "pavan",
    "email": "pavan@example.com"
  },
  {
    "id": "3",
    "name": "pamidi",
    "email": "pamidi@example.com"
  }
]
```

Output for Question 4

Q7: Node.js Core Modules – Logging & HTTP Server

Details

- CLI Node script.
- Writes App started into logs/app.log using fs.
- Starts a simple HTTP server returning { "status": "running" }.
- Uses only Node.js core modules (no external dependencies).

Features

- File logging.
- Simple JSON HTTP response.
- No NPM packages required.

Q8: Asynchronous JS – Callbacks → Promise → Async/Await

Details

- Refactor callback-based fetchData into Promise and Async/Await.

- Demonstrates modern async patterns.

Features

- Maintain same output.
- Shows three versions: callback, promise, async/await.

Q9: Express Routing + Middleware + Validation

Details

- Express server with /products GET and POST.
- Middleware logs method + URL.
- POST validates name and price using express-validator.

Features

- GET returns all products.
- POST validates and returns JSON.
- Middleware logs request info.

Q10: REST API + JWT Authentication

Details

- Login route /login returns JWT for static user.
- Protected route /dashboard accessible only with valid JWT.

Features

- JWT token generation & validation.
- Express middleware for protection.
- Demonstrates authentication workflow.

Folder Structure

/backend

 /routes

 products.js # Q9

 /middleware

 logger.js # Q9

auth.js # Q10
logs/ # Q7
server.js # Q7/Q9/Q10 combined
package.json

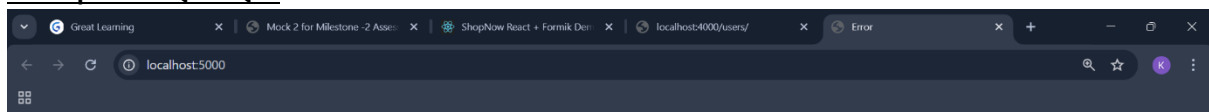
Execution

>cd backend
>npm install express express-validator jsonwebtoken
>node server.js

#Try

><http://localhost:5000/status>
><http://localhost:5000/async-demo>
><http://localhost:5000/products>

output for Q7 – Q10



Cannot GET /



